Chuvadi.Sheets 2.0 — The Guide

சுவடி · *Chuvadi* — palm-leaf manuscript, the original document format of South India.

A pure-BCL spreadsheet (xlsx) library for .NET 10: write, read, style, validate, protect and encrypt Excel workbooks with no NuGet dependencies.

Item	Value
Library version	Chuvadi.Sheets 2.0.1 (2.0.0 released 2026-11-05)
Namespace	`Chuvadi.Sheets.Model`
Target	.NET 10
Author	Dr. Arun Shiva B
Source	github.com/arunshivab/chuvadi
Licence	MIT. Library and documentation © 2026 Arun Shiva B.

This guide is written in Markdown (`docs/Chuvadi.Sheets/GUIDE.md` in the repository) and rendered to PDF with Chuvadi's own tools: Chuvadi.Layout.Markup lays it out, Chuvadi.Layout.Pdf writes the PDF, and every font in it is an SIL Open Font License font that ships with Chuvadi.Layout.Fonts or Chuvadi.Fonts. Every C# example in it is compiled and run against the library before the PDF is published (see the last chapter).

Contents

Part	Chapters
Part I · Getting started	1 Introduction (3) · 2 Installation (5) · 3 First steps (7) · 4 Upgrading from 1.x (9)
Part II · Writing workbooks	5 Choosing a write API (11) · 6 The Workbook model (12) · 7 The streaming writer (14) · 8 Values and types (17) · 9 Cell styles (19) · 10 Formulas and defined names (21) · 11 One-line exports (22) · 12 Sheet features (24) · 13 Page setup and printing (26) · 14 Data validation (28) · 15 Conditional formatting (29)
Part III · Reading workbooks	16 The streaming reader (31) · 17 Loading a workbook (34) · 18 Typed and table reads (35) · 19 Display text and number formats (36) · 20 Styles on read (37) · 21 Layout for renderers (38) · 22 Rich text, hyperlinks and comments (39) · 23 Drawings, charts and header pictures (40)
Part IV · Security	24 Sheet and workbook protection (41) · 25 Encryption (42) · 26 Untrusted input (44) · 27 Errors (46)
Part V · Operations and reference	28 Performance (47) · 29 Threading (48) · 30 The zip API (obsolete) (49) · 31 Options reference (50) · 32 Public API map (52) · 33 Troubleshooting (54) · 34 About this guide (55)

1 Introduction

1.1 What Chuvadi.Sheets is

Chuvadi.Sheets reads and writes Office Open XML spreadsheets (`.xlsx`) from .NET 10. It is one assembly, `Chuvadi.Sheets.dll`, built only on the .NET base class library: the package declares no dependencies, and continuous integration fails the build if a package reference ever appears in the library's project.

It offers three ways to write, chosen by how much data you have and how much control you need, and two ways to read:

Task	API	When
Write, cell by cell, in any order	`Workbook`	Reports and forms of ordinary size; load, edit and save
Write, row by row, in bounded memory	`XlsxWriter`	Exports of any size, 100,000 rows and far beyond
Write a list or table in one line	`.ToXlsx()`, `XlsxExport`	Lists of objects and `DataTable`s
Read, row by row, in bounded memory	`XlsxReader`	Imports, validation, large files
Read into a model	`Workbook.Load`	Inspect or edit a whole workbook, styles included
Read into objects or tables	`.ReadXlsx<T>()`, `.ReadXlsxSheet()`	Typed imports

Around these sit the features real workbooks need: cell styles and number formats, formulas with cached results, defined names, tables, merges, freeze panes, autofilters, hyperlinks, comments, rich text, data validation, conditional formatting, page setup with headers and footers, column auto-fit, sheet and workbook protection, and workbook encryption that Excel opens with a password.

On the read side it goes further than values. It resolves every cell's style, renders a value as Excel displays it, evaluates conditional formats per cell, and reports drawings, charts, comments and print settings. That makes it a sound base for viewers and converters (Chuvadi.Layout.Sheets lays out and renders sheets on top of it).

1.2 Design principles

- **Pure BCL.** No NuGet packages, no native binaries. Nothing to vet or upgrade separately.
- **Streaming first.** The streaming writer and reader use bounded memory whatever the row count.
- **Excel as ground truth.** Files must open in Excel without repair prompts; encrypted files open in Excel with their password; display text matches what Excel shows.
- **Never alter data.** Values are read and written exactly as given. What the model cannot keep is reported (`Workbook.LoadWarnings`), never silently changed.
- **Untrusted input is a first-class case.** Read limits, cancellation and a single exception type for malformed files make it safe to open uploads.

1.3 What it does not do

- It does not calculate formulas. It writes them, with a cached result if you give one, and reads them with the result the file stores.
- It reads drawings, pictures, charts, chart sheets and header/footer pictures, but does not write them.
- No pivot tables, no custom table styles, no macros (macro-enabled files are detected, never run, and saved without macros), no zip encryption.
- Not every style property it reads is written back on load and save. Chapter 20 lists what round-trips.

2 Installation

2.1 Requirements

- .NET 10 SDK to build; .NET 10 runtime to run.
- Any operating system .NET 10 supports. The library has no platform-specific code; continuous integration runs on Linux and Windows.

2.2 Getting the package

Chuvadi.Sheets is published as a NuGet package attached to its GitHub release, not on nuget.org. Download `Chuvadi.Sheets.2.0.1.nupkg` from the release page (tag `sheets-v2.0.1`), put it in a folder, and use that folder as a package source.

```
New-Item -ItemType Directory -Force -Path C:\local-nuget | Out-Null
Copy-Item .\Chuvadi.Sheets.2.0.1.nupkg C:\local-nuget\
Get-FileHash C:\local-nuget\Chuvadi.Sheets.2.0.1.nupkg -Algorithm SHA256
```

The release page shows the package's SHA-256 digest; the hash printed here should be the same.

Add a `nuget.config` next to your solution so the folder is a source:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <add key="local" value="C:\local-nuget" />
  </packageSources>
</configuration>
```

Then reference the package:

```
dotnet add package Chuvadi.Sheets --version 2.0.1
```

2.3 Building from source

```
git clone https://github.com/arunshivab/chuvadi
Set-Location chuvadi
dotnet build src\Chuvadi.Sheets -c Release
dotnet test tests\Chuvadi.Sheets.Tests -c Release
```

The verification suite writes inspectable files you can open in Excel:

```
dotnet run -c Release --project samples\Chuvadi.Sheets.ManualTests
```

2.4 Namespaces

Namespace	Contents
`Chuvadi.Sheets.Model`	Everything for xlsx: writers, readers, the model, styles, options, exceptions

Namespace	Contents
`Chuvadi.Sheets.Zip`	The old zip API: obsolete since 2.0.0, removed in the next major version (chapter 30)

One `using Chuvadi.Sheets.Model;` is all the examples in this guide need.

3 First steps

3.1 Write a workbook

```
var wb = new Workbook();
var sheet = wb.AddSheet("Patients");

var header = new CellStyleBuilder().Bold().Background("DDEBF7").Build();
sheet.Cell("A1").Value = "Name";        sheet.Cell("A1").Style = header;
sheet.Cell("B1").Value = "Admitted";    sheet.Cell("B1").Style = header;
sheet.Cell("C1").Value = "Bill";        sheet.Cell("C1").Style = header;
sheet.Cell("D1").Value = "With tax";    sheet.Cell("D1").Style = header;

sheet.Cell("A2").Value = "Asha Rao";
sheet.Cell("B2").Value = new DateTime(2026, 3, 14);             // shown as 2026-03-14
sheet.Cell("C2").Value = 1234.5;
sheet.Cell("C2").Style = CellStyle.Default with { NumberFormat = "#,##0.00" };

sheet.Cell("D2").Formula = "C2*1.18";
sheet.Cell("D2").CachedValue = 1456.71;    // shown by readers that do not calculate
sheet.Cell("D2").Style = CellStyle.Default with { NumberFormat = "#,##0.00" };

sheet.FreezeRows(1);
sheet.AutoFitColumns();
wb.SaveTo("first.xlsx");
```

The model keeps cells in a sparse dictionary, so you may set them in any order. Dates are stored as Excel date serials and get a date format automatically when you give no style.

3.2 Read it back

```
using var reader = XlsxReader.Open("first.xlsx");
foreach (var row in reader.Sheet("Patients").Rows)
{
    string name = row.GetString("Name");
    DateTime admitted = row.GetDateTime("Admitted");
    string bill = row.GetDisplayString("Bill");                  // "1,234.50", as Excel shows it
    string? formula = row.GetFormula("With tax");               // "C2*1.18"
    Console.WriteLine($"{name}: {admitted:yyyy-MM-dd}, {bill}, {formula} = {row["With tax"]}");
}
```

By default the first row is the header row: it is not returned as data, and its text names the columns for `GetString("Name")` and friends. Set `TreatFirstRowAsHeaders = false` in `XlsxReaderOptions` to read every row and address columns by 0-based index.

Note the two kinds of value a reader returns. `row["Bill"]` is the raw value, a `double` 1234.5; `row.GetDisplayString("Bill")` is that value rendered with the cell's number format, the text a person sees.

3.3 Stream a large export

```
using (var writer = XlsxWriter.Create("large.xlsx"))
{
    using var sheet = writer.AddSheet("Data");
    sheet.WriteHeader("Id", "Name", "Amount");
    for (int i = 1; i <= 100_000; i++)
        sheet.WriteRow(i, "Customer " + i, i * 1.5);
    writer.Save();
}
```

Rows go to a temporary file as they are written, so memory stays flat however many rows there are; `Save` assembles the package. Calling `Save` is required: disposing the writer alone produces no file, and leaves an existing file of that name as it was.

3.4 Export a list in one line

```
var visits = new List<Visit>
{
    new() { Id = 1, Patient = "Asha Rao",  Date = new DateTime(2026, 3, 14), Fee = 500m },
    new() { Id = 2, Patient = "Ravi Iyer", Date = new DateTime(2026, 3, 15), Fee = 750m },
};
visits.ToXlsx("visits.xlsx", cfg => cfg
    .SheetName("Visits")
    .AsTable("VisitsTable")
    .FreezeHeaderRow()
    .AutoFitColumns());

List<Visit> back = XlsxReadExtensions.ReadXlsx<Visit>("visits.xlsx");
Console.WriteLine($"{back.Count} visits read back");
```

```
public sealed class Visit
{
    public int Id { get; set; }
    public string Patient { get; set; } = "";
    [Column("Visit date", Format = "dd-mmm-yyyy")]
    public DateTime Date { get; set; }
    [Column(Format = "#,##0.00")]
    public decimal Fee { get; set; }
}
```

The column headers come from the property names, or from `[Column]`; `ReadXlsx<T>` maps them back by the same header text.

4 Upgrading from 1.x

Chuvadi.Sheets 2.0.0 is a major version with three breaking changes; 2.0.1 is a bug-fix release on top of it (section 4.5). Values, styles, display text and saved output are the same as in 1.8.0; the public API is the same apart from the namespace and the obsolete zip types.

4.1 The namespace is now `Chuvadi.Sheets.Model`

`Chuvadi.Sheets.Excel` was renamed to `Chuvadi.Sheets.Model`, matching `Chuvadi.Docs.Model`. Every type keeps its name and members.

- Replace `using Chuvadi.Sheets.Excel;` with `using Chuvadi.Sheets.Model;`, and any fully qualified `Chuvadi.Sheets.Excel.` name the same way.
- Recompile. Code built against 1.x does not run against 2.0, because the type names changed.

4.2 One exception for malformed files

Every error caused by a malformed file is now `XlsxFormatException`, with the original exception as `InnerException`:

The file has	1.x threw	2.0 throws
No zip, a damaged or truncated zip, a missing part	`InvalidDataException`	`XlsxFormatException` (`InvalidDataException` inside)
Malformed XML in a part (including with `StrictXml`)	`XmlException`	`XlsxFormatException` (`XmlException` inside)
A damaged relationship entry, such as an empty `Id`	`ArgumentException`	`XlsxFormatException` (`ArgumentException` inside)
A failed integrity check of an encrypted file	`InvalidDataException`	`XlsxFormatException` (`InvalidDataException` inside)
A damaged encrypted container, with a password given (bug fix)	`XlsxPasswordRequiredException`	`XlsxFormatException`

Unchanged: a missing or wrong password is `XlsxPasswordRequiredException`; read limits are `XlsxLimitExceededException`, except `MaxPartBytes`, which is still `InvalidDataException`; cancellation is `OperationCanceledException`.

To migrate, replace `catch (XmlException)` and `catch (InvalidDataException)` around reads with `catch (XlsxFormatException)`, and keep `catch (InvalidDataException)` only for `MaxPartBytes`. Chapter 27 has the full list.

4.3 `Chuvadi.Sheets.Zip` is obsolete

The zip API (`ZipWriter`, `ZipReader`, `ZipFormatException`, the zip extension methods) produces warning CS0618 and will be removed in the next major version. Use the Chuvadi.Zip package instead. `ZipFormatException` was marked in 2.0.1; in 2.0.0 it was left unmarked because Chuvadi.Layout.Sheets still named it.

4.4 Also new in 2.0.0

- Saving many relationships (hyperlinks, pictures) is linear: 50,000 hyperlinks save in about 0.4 s instead of 6.8 s.
- The `InvalidDataException` thrown for `MaxPartBytes` carries `Data["Chuvadi.Sheets.ReadLimit"] = true`, so a size limit can be told apart from a damaged zip (chapter 26).

4.5 Changes in 2.0.1

2.0.1 fixes bugs that 2.0.0 and every 1.x release share. The public API is the same; only these behaviours change.

What	2.0.0	2.0.1
Encrypted files larger than about 7 MB	Saved without error, but no reader could open them (bug fix)	Written correctly; checked with this library, msoffcrypto-tool and LibreOffice (chapter 25)
Saving to a stream	Closed the stream, although documented as left open (bug fix)	Left open, after the written bytes (section 6.4)
An encrypted save to a stream that already holds bytes	Overwrote them from the start (bug fix)	Written from the stream's position, after them
An encrypted save to a non-seekable stream	`NotSupportedException` (bug fix)	Works, like an unencrypted save
Saving to a path that exists	The file was emptied first, so a failed save destroyed it (bug fix)	Written beside it and replaced only when the save succeeds (section 7.6)
Temporary files of an encrypted save	The package and sheet data in plaintext (bug fix)	Encrypted with a key held only in memory (section 7.6)
Temporary files on Linux and macOS	Readable by other users with the usual umask (bug fix)	Readable by the current user only
`ZipFormatException`	Not marked obsolete	Obsolete, warning CS0618 (section 4.3)

With the safe replace, a path save creates or replaces the file only at `Save`: until then the output goes to `.<name>.<8 hex digits>.tmp` in the same folder. Everything else is as before: a missing folder, a read-only or locked file, or a folder of that name fails at `XlsxWriter.Create` (or `SaveTo`) with the same exception; a replaced file keeps its permissions (and on Windows its attributes and security settings); and a symbolic link is kept, with the file it points to replaced.

5 Choosing a write API

All three write APIs produce the same kind of file and share the same styles, formats and sheet features. They differ in how you hand over the data.

	`Workbook`	`XlsxWriter`	`.ToXlsx()` / `XlsxExport`
Order of writing	Any cell, any order	Row by row, top to bottom	A whole list or table at once
Memory	Proportional to populated cells	Flat (rows spool to a temp file)	Flat (built on `XlsxWriter`)
Reads existing files	Yes: `Workbook.Load`, edit, save	No	No
Encryption	`SaveTo(path, EncryptionOptions)`	`XlsxWriterOptions.Encryption`	Through `XlsxWriter`
Best for	Forms, summaries, templates, load-edit-save	Exports of any size	Quick exports of objects and `DataTable`s

A rule of thumb: use `XlsxWriter` when the row count is large or unknown, `Workbook` when you need to place cells freely or edit a file, and the one-liners when the data already is a list.

The model fills row gaps with empty row elements when it saves: a sheet with cells only in A1 and A100000 still writes 100,000 rows. It is meant for ordinary, dense sheets; for very large or very sparse data use the streaming writer.

6 The Workbook model

6.1 Sheets and cells

```
var wb = new Workbook();
var summary = wb.AddSheet("Summary");
var data = wb.AddSheet("Data");

data.Cell("A1").Value = "Region";               // by address
data.Cell(1, 2).Value = "Sales";                // by row and column, both 1-based
data.Cell(2, 1).Value = "North";
data.Cell(2, 2).Value = 125000;

Console.WriteLine(data.Cell(1, 2).Address);     // "B1"
Console.WriteLine(wb["data"].Name);             // case-insensitive lookup: "Data"

wb.ActiveSheet = data;                          // the tab Excel opens on
wb.SaveTo("model.xlsx");
```

`Sheet.Cell(...)` creates the cell if it does not exist yet. To look a cell up without creating it, use `FindCell(row, column)` or `FindCell("B2")`, which return `null` for an untouched cell. `PopulatedCells` lists the cells that hold something.

A `Cell` carries:

Property	Meaning
`Value`	The value: `string`, a number, `bool`, `DateTime`, `TimeSpan`, … (chapter 8), or `null`
`Formula`, `CachedValue`	A formula without the leading `=`, and its last result for readers that do not calculate
`ArrayFormulaRange`	For an array formula, the range it fills
`RichText`	A string made of runs with their own fonts (chapter 22)
`Style`	The cell's `CellStyle`, or `null` for the default
`DisplayText`	The value as Excel shows it with the cell's number format
`Row`, `Column`, `Address`	Where it is

6.2 Ranges, merges and sizes

```
var wb = new Workbook();
var sheet = wb.AddSheet("Report");

sheet.Cell("A1").Value = "Quarterly report";
sheet.Range("A1:D1").Merge();                   // or sheet.MergeCells("A1:D1")

sheet.Columns[1].Width = 28;                    // characters of the default font
sheet.Columns[4].Hidden = true;
sheet.Rows[1].Height = 30;                      // points

sheet.FreezeRows(1);
sheet.FreezeColumns(1);
sheet.AutoFilter("A2:D50");
sheet.TabColor = "1F4E79";
wb.SaveTo("ranges.xlsx");
```

`sheet.Columns[n]` and `sheet.Rows[n]` are 1-based and settable: width, hidden, outline level, collapsed and a style for the whole column or row. Chapter 12 covers views, outlines and visibility in full.

6.3 Load, edit, save

```
var wb = Workbook.Load("model.xlsx");
var data = wb["Data"];
data.Cell(3, 1).Value = "South";
data.Cell(3, 2).Value = 98000;
wb.SaveTo("model-edited.xlsx");
```

A load-edit-save keeps values, formulas with their cached results, defined names, page setup, merges, row and column settings, the sheet view, tab colour, visibility, the active sheet, conditional formats, data validation, rich text, hyperlinks and comments, plus the style properties the writer supports. It does not keep tables, charts, pivots, drawings or the style properties chapter 20 lists. Formulas that refer to other workbooks, to tables or to names the model could not keep are saved as their cached values.

6.4 Saving to a stream

```
var wb = new Workbook();
wb.AddSheet("Data").Cell("A1").Value = "Hello";

var ms = new MemoryStream();
wb.SaveTo(ms);                                  // the stream stays open, positioned after the file
Console.WriteLine($"{ms.Length:N0} bytes, position {ms.Position:N0}");

ms.Position = 0;                                // read it back from the same stream
Console.WriteLine(Workbook.Load(ms)["Data"].Cell("A1").Value);
```

The stream is flushed and left open: the save writes from the stream's current position, leaves the position after the last byte written, and never closes it, so you can keep using it or hand it on, as with an HTTP response body. The stream need not be seekable. The same holds for `XlsxWriter.Create(stream)`, `ToXlsx(stream)` and `new XlsxExport(stream)`, encrypted or not. (2.0.0 and earlier closed the stream after an unencrypted save, although the documentation said it did not.)

7 The streaming writer

7.1 The shape of a streamed export

```
using (var writer = XlsxWriter.Create("export.xlsx"))
{
    using (var sheet = writer.AddSheet("Orders"))
    {
        sheet.SetColumnWidth(1, 10);
        sheet.SetColumnWidth(2, 3, 18);          // columns 2 to 3
        sheet.FreezeRows(1);
        sheet.WriteHeader("Order", "Customer", "Placed", "Total");
        sheet.WriteRow(1001, "Asha Rao", new DateTime(2026, 4, 1), 2500.0);
        sheet.WriteRow(1002, "Ravi Iyer", new DateTime(2026, 4, 2), 1250.5);
        sheet.AutoFilter("A1:D3");
    }
    using (var notes = writer.AddSheet("Notes"))
        notes.WriteRow("Generated by the nightly job");
    writer.Save();
}
```

- `XlsxWriter.Create` takes a path (created or overwritten) or a stream (left open), and optional `XlsxWriterOptions`.
- `AddSheet` returns a `SheetWriter`. Fill sheets one after another and dispose each one (a `using` block); `Save` finalizes any sheet that is still open.
- Rows are written top to bottom. Sheet settings (widths, freeze panes, filters, merges, page setup, protection, validation, conditional formats, tables) may be set at any time before `Save`.
- `Save` assembles the package and writes it to the output. After `Save` the writer accepts no more changes.

7.2 Per-cell control with `RowBuilder`

`WriteRow(params object[])` types each value at run time. For styles, formulas, links and comments per cell, build the row:

```
var money = CellStyle.Default with { NumberFormat = "#,##0.00" };
var bold = new CellStyleBuilder().Bold().Build();

using var writer = XlsxWriter.Create("builder.xlsx");
using var sheet = writer.AddSheet("Invoice");
sheet.WriteHeader("Item", "Qty", "Price", "Amount", "Link", "Note");
sheet.WriteRow(r => r
    .Cell("Consultation")
    .Cell(2)
    .Cell(500.0, money)
    .Formula("B2*C2", 1000.0, money)             // formula plus its cached result
    .Hyperlink("https://example.com/item/1", "details")
    .Comment("see note", "Asha", "Discount approved"));
sheet.WriteRow(r => r
    .Cell("Total", bold)
    .Empty()                                     // skip a column
    .Empty()
    .Formula("SUM(D2:D2)", 1000.0, money));
writer.Save();
```

`RowBuilder` method	Writes
`Cell(value)`, `Cell(value, style)`	A value, typed at run time (chapter 8)

`RowBuilder` method	Writes
`Formula(f)`, `Formula(f, cached)`, `Formula(f, cached, style)`	A formula without the `=`; the cached result is optional
`ArrayFormula(range, f, cached)`	The top-left cell of an array formula filling `range`
`Hyperlink(url, display, tooltip)`	A linked cell in the default hyperlink style (blue, underlined)
`Comment(value, author, text)`	A value with a note attached
`Empty()`	Nothing; moves to the next column

7.3 Row and column settings

```
using var writer = XlsxWriter.Create("formats.xlsx");
using var sheet = writer.AddSheet("Grouped");

sheet.SetColumns(1, 1, new ColumnFormat { Width = 30 });
sheet.SetColumns(3, 4, new ColumnFormat { Width = 12, OutlineLevel = 1, Hidden = true });

sheet.WriteHeader("Department", "Total", "Q1", "Q2");
sheet.WriteRow(new RowFormat { Height = 24, Style = new CellStyleBuilder().Bold().Build() },
               "Radiology", 1500, 700, 800);
sheet.WriteRow(new RowFormat { OutlineLevel = 1 }, "  Imaging", 900, 400, 500);
sheet.WriteRow(new RowFormat { Hidden = true });  // an empty row that carries only its settings
writer.Save();
```

Spans set with `SetColumns` and `SetColumnWidth` are written exactly as given, in the order of the calls; overlapping spans are neither merged nor rejected. Applications differ on the columns two spans share. After `SetColumnWidth(1, 3, 5)` and then `SetColumnWidth(2, 4, 30)`, `Workbook.Load` applies the later span (widths 5, 30, 30, 30) and LibreOffice 24.2 the earlier one (5, 5, 5, 30). For one width per column everywhere, do not overlap spans. A span with a width wins over auto-fit for its columns.

7.4 Auto-fit

```
using var writer = XlsxWriter.Create("autofit.xlsx");
using var sheet = writer.AddSheet("Fit");
sheet.AutoFitColumns(new AutoFitOptions { MaxWidth = 50 });   // before the first row
sheet.WriteHeader("Code", "Description");
sheet.WriteRow("A-1", "A short line");
sheet.WriteRow("B-22", "A much longer description that sets the column width");
writer.Save();
```

The writer measures every cell as displayed (number format, font, size, bold) and writes the widths at save time. Call `AutoFitColumns` before writing rows, so every row is measured. Wrapped, merged and rotated cells are not measured; formula cells are measured by their cached result. Widths set with `SetColumnWidth` or `SetColumns` win. Measurement uses built-in width tables for Calibri, Cambria, Arial, Times New Roman and Courier New and their metric-compatible equivalents, so it needs no fonts installed.

`AutoFitOptions`: `MinWidth`, `MaxWidth`, `MaxRowsToMeasure`, `ExtraWidth` (padding), `AllowForFilterButton`, `Columns` (only these, 1-based) and `Format` (culture and date system for the displayed text).

7.5 Asynchronous writing

```
await using (var writer = XlsxWriter.Create("async.xlsx"))
{
    await using (var sheet = writer.AddSheet("Data"))
    {
        await sheet.WriteHeaderAsync("Id", "Value");
        for (int i = 1; i <= 1000; i++)
            await sheet.WriteRowAsync(i, i * i);
    }
    await writer.SaveAsync();
}
```

The asynchronous methods write the same XML with non-blocking I/O. They do not add concurrency: do not run two calls against the same writer at the same time.

7.6 Temporary files and inline strings

- Each sheet spools to a temporary file named `chuvadi_sheets_*.tmp` in `Path.GetTempPath()` or in `XlsxWriterOptions.TempDirectory`. Each one is opened exclusively, readable by the current user only on Linux and macOS, and deleted when it is copied into the package, when the writer is disposed, or by the operating system if the process ends.
- Strings go through a shared strings table by default, which keeps files small when strings repeat. At save time the sheet files are streamed into the package in one pass, with each string's placeholder replaced by its final index.
- `UseInlineStrings = true` writes each string in its cell instead: no shared strings table and no post-processing pass, at the cost of larger files when strings repeat.
- An encrypted save (chapter 25) also spools the package before encrypting it. For an encrypted save every temporary file, the sheet spools included, is itself encrypted (AES-256-CTR) with a one-time key held only in memory, so neither the package nor the data reaches the disk in plaintext, even if the process crashes.
- A save to a path writes `.<name>.<8 hex digits>.tmp` beside the target and replaces the target only when the save succeeds: a failed or cancelled save, or a writer disposed without `Save`, leaves an existing file as it was and removes the temporary file. A replaced file keeps its permissions (on Windows also its attributes and security settings), and a symbolic link stays a link.

8 Values and types

8.1 Writing

Both writers type a value by its .NET type:

.NET type	Stored as	Default number format when no style is given
`string`	Shared string (or inline string)	General
`bool`	Boolean cell	General
`int`, `long`, `short`, `byte`, `uint`, `ulong`, `sbyte`, `ushort`	Number	General
`double`, `float`, `decimal`	Number (`decimal` and `float` converted to `double`)	General
`DateTime` without a time of day, `DateOnly`	Date serial	`DefaultDateFormat`, `yyyy-mm-dd`
`DateTime` with a time of day	Date serial with fraction	`DefaultDateTimeFormat`, `yyyy-mm-dd hh:mm:ss`
`DateTimeOffset`	Date serial of its clock time (the offset is not stored)	As `DateTime`
`TimeSpan`	Fraction of days	`DefaultTimeSpanFormat`, `[h]:mm:ss`
`RichText`	Rich string with runs	General
`null`	Empty cell (a style still applies)	—
Anything else	String, from `ToString()`	General

The default formats are set in `XlsxWriterOptions`, so a date never shows as a bare serial number:

```
var options = new XlsxWriterOptions
{
    DefaultDateFormat = "dd/mm/yyyy",
    DefaultDateTimeFormat = "dd/mm/yyyy hh:mm",
    DefaultTimeSpanFormat = "[h]:mm",
};
using var writer = XlsxWriter.Create("dates.xlsx", options);
using var sheet = writer.AddSheet("Dates");
sheet.WriteRow(new DateTime(2026, 5, 1),
               new DateTime(2026, 5, 1, 9, 30, 0),
               TimeSpan.FromHours(30));
writer.Save();
```

Excel stores dates as serial numbers in the 1900 date system; the writer always saves in that system.

8.2 Reading

Stored	Returned by the readers
String (shared, inline or formula string)	`string`

Stored	Returned by the readers
Number	`double`
Number with a date or time format	`DateTime` (with `AutoDetectDates`, the default); otherwise `double`
Boolean	`bool`
Error (`#N/A`, `#DIV/0!`, …)	`string`, the error text
Empty or missing	`null`

Files in the 1904 date system are converted on read (`Workbook.Date1904` and `XlsxReader.Date1904` say which). The typed getters convert for you: `GetInt32`, `GetInt64`, `GetDecimal`, `GetDouble`, `GetBool`, `GetDateTime`, `GetString` and `Get<T>`; `TryGet<T>` returns `false` instead of throwing (chapter 16).

9 Cell styles

9.1 `CellStyle` and the builder

`CellStyle` is an immutable value: equal styles are interchangeable and may be used as dictionary keys. Build one with `CellStyleBuilder`, or start from `CellStyle.Default` with a `with` expression.

```
var header = new CellStyleBuilder()
    .Font("Calibri", 12)
    .Bold()
    .Foreground("FFFFFF")                       // font colour, "RRGGBB"
    .Background("1F4E79")                       // solid fill
    .BorderAll(BorderStyle.Thin, "BFBFBF")
    .HAlign(HorizontalAlign.Center)
    .VAlign(VerticalAlign.Center)
    .WrapText()
    .Build();

var money = CellStyle.Default with { NumberFormat = "#,##0.00", HAlign = HorizontalAlign.Right };
var warning = money with { FontColor = "C00000", Bold = true };

var wb = new Workbook();
var sheet = wb.AddSheet("Styles");
sheet.Cell("A1").Value = "Amount";  sheet.Cell("A1").Style = header;
sheet.Cell("A2").Value = 1234.5;    sheet.Cell("A2").Style = money;
sheet.Cell("A3").Value = -50.0;     sheet.Cell("A3").Style = warning;
wb.SaveTo("styles.xlsx");
```

9.2 What the writer writes

The writer writes these properties:

Group	Properties
Font	`FontName`, `FontSize`, `Bold`, `Italic`, `Underline`, `FontColor`
Fill	`FillColor` (a solid fill)
Borders	`BorderTop`, `BorderBottom`, `BorderLeft`, `BorderRight`: each a `Border(style, colour)` with any of the 14 `BorderStyle`s
Number format	`NumberFormat`, any Excel format code
Alignment	`HAlign`, `VAlign`, `WrapText`

`CellStyle` has more properties, added in 1.2.0 for reading: underline kind, strikethrough, superscript and subscript, pattern and gradient fills, diagonal borders, indent, rotation, shrink-to-fit, reading order, cell protection, quote prefix and named styles. They are filled in when a file is read (chapter 20) and are not written.

Colours are six hex digits, `RRGGBB`; a leading `#` is accepted.

9.3 Number formats

Any Excel format code works: `0.00`, `#,##0`, `0%`, `0.00E+00`, `dd-mmm-yyyy`, `hh:mm`, `[h]:mm:ss`, `"Rs." #,##0.00`, `#,##0;[Red]-#,##0`, `@` (text). A value keeps its type; the format only changes how it shows. `NumberFormatter.Format(value, code)` renders a value with a code exactly as Excel would, without writing a file (chapter 19).

9.4 Column styles from attributes

For one-line exports, a style can be declared on the property (chapter 11):

```
public sealed class WorkItem
{
    public string Title { get; set; } = "";
    [ColumnStyle(Bold = true, Background = "FFEB3B")]
    public string Status { get; set; } = "";
}
```

10 Formulas and defined names

10.1 Formulas

Formulas are written without the leading `=`. The library does not calculate them: Excel does when the file is opened. If you know the result, store it as the cached value, so that viewers, converters and this library's readers can show it.

```
var wb = new Workbook();
var sheet = wb.AddSheet("Calc");
sheet.Cell("A1").Value = 120;
sheet.Cell("B1").Value = 18;
sheet.Cell("C1").Formula = "A1*B1/100";
sheet.Cell("C1").CachedValue = 21.6;
wb.SaveTo("calc.xlsx");

var back = Workbook.Load("calc.xlsx")["Calc"].Cell("C1");
Console.WriteLine($"{back.Formula} = {back.CachedValue}");   // "A1*B1/100 = 21.6"
```

On the streaming writer use `RowBuilder.Formula(f, cachedValue)` and `ArrayFormula(range, f, cachedValue)`. A cached value may be a number, string, `bool`, date, `TimeSpan`, or a string equal to an Excel error such as `#N/A`, which is stored as that error.

When cached values may be approximate, set `XlsxWriterOptions.FullCalculationOnLoad = true`: the workbook then tells Excel to recalculate every formula on open instead of trusting stored results.

On read, `row.GetFormula(col)` and `cell.Formula` give the formula text; shared formulas are expanded to each cell's own formula. `row[col]` and `cell.CachedValue` give the stored result.

10.2 Defined names

```
var wb = new Workbook();
var rates = wb.AddSheet("Rates");
rates.Cell("B2").Value = 0.18;
wb.DefinedNames.Add("TaxRate", "Rates!$B$2");                              // workbook scope
wb.DefinedNames.Add("Local", "Rates!$B$2*2", "Rates", hidden: false, comment: "sheet-scoped");
wb.SaveTo("names.xlsx");

using var writer = XlsxWriter.Create("names-stream.xlsx");
using (var sheet = writer.AddSheet("Data")) sheet.WriteRow(1, 2, 3);
writer.AddDefinedName("DataRow", "Data!$A$1:$C$1");
writer.Save();
```

The formula or reference is given without `=`. A name added twice with the same scope replaces the first. Print areas and print titles are not added as names: set them on the sheet's `PageSetup` (chapter 13). On read, `XlsxReader.DefinedNames` and `Workbook.LoadedDefinedNames` list every name exactly as stored, including `_xlnm.Print_Area` and `_xlnm.Print_Titles`.

11 One-line exports

11.1 Lists of objects

`ToXlsx` writes one sheet from any `IEnumerable<T>`, one column per public property. Configure it with attributes, with the lambda, or both; the lambda wins.

```
public sealed class Patient
{
    [Column("Patient ID", Order = 1, Width = 14)]
    public string Id { get; set; } = "";

    [Column("Name", Order = 2)]
    public string Name { get; set; } = "";

    [Column("Date of birth", Order = 3, Format = "dd-mmm-yyyy")]
    public DateTime DateOfBirth { get; set; }

    [ColumnStyle(Bold = true)]
    public string Ward { get; set; } = "";

    [ColumnIgnore]
    public string InternalNotes { get; set; } = "";
}
```

```
var patients = new[]
{
    new Patient { Id = "P-001", Name = "Asha Rao",
                  DateOfBirth = new DateTime(1980, 1, 5), Ward = "A" },
    new Patient { Id = "P-002", Name = "Ravi Iyer",
                  DateOfBirth = new DateTime(1975, 7, 9), Ward = "B" },
};

patients.ToXlsx("patients.xlsx", cfg => cfg
    .SheetName("Patients")
    .HeaderStyle(new CellStyleBuilder().Bold().Background("DDEBF7").Build())
    .AsTable("PatientsTbl", TableStyle.Medium2)
    .FreezeHeaderRow()
    .AutoFitColumns()
    .Column(p => p.Name, header: "Full name")
    .PageHeaderFooter("&CPatient list", "&LConfidential&RPage &P of &N")
    .PageSetup(p => { p.Orientation = PageOrientation.Landscape; p.FitTo(1, 0); }));
```

Configuration	Effect
`SheetName(name)`	Sheet name (default `Sheet1`)
`HeaderStyle(style)`	Style of the header row
`AutoFilter()`, `FreezeHeaderRow()`	Filter buttons on the header; header stays visible
`AsTable(name, style)`	Wraps the data in an Excel table
`AutoFitColumns(options)`	Sizes columns to their content; explicit widths win
`Column(selector, header, width, format, style)`	Overrides one column
`Ignore(selector)`, `ColumnStyle(selector, style)`	Leaves a column out; styles a column's data cells
`PageHeaderFooter(header, footer)`, `PageSetup(p => ...)`	Printing (chapter 13)

`[Column]` sets the header, order, width and format; `[ColumnStyle]` sets bold, italic, underline, font colour (`Foreground`), fill (`Background`) and format; `[ColumnIgnore]` leaves a property out.

11.2 DataTables

```
var table = new DataTable("Visits");
table.Columns.Add("Id", typeof(int));
table.Columns.Add("Patient", typeof(string));
table.Columns.Add("Fee", typeof(decimal));
table.Rows.Add(1, "Asha Rao", 500m);
table.Rows.Add(2, "Ravi Iyer", 750m);

table.ToXlsx("table.xlsx", cfg => cfg
    .ColumnFormat("Fee", "#,##0.00")
    .AutoFilter()
    .AutoFitColumns());
```

The `DataTable` configuration has the same settings, addressed by column name (`Column`, `Ignore`, `ColumnStyle`, `ColumnFormat`). The sheet name defaults to the table's name.

11.3 Several sheets

```
var visits = new DataTable("Visits");
visits.Columns.Add("Id", typeof(int));
visits.Rows.Add(1);

var patients = new[] { new Patient { Id = "P-001", Name = "Asha Rao", Ward = "A" } };

new XlsxExport("multi.xlsx")
    .AddSheet("Patients", patients, cfg => cfg.AutoFilter().FreezeHeaderRow())
    .AddSheet("Visits", visits)
    .Save();
```

`XlsxExport` also takes a stream. The one-liners are built on the streaming writer, so memory stays flat for large lists.

12 Sheet features

12.1 Views, panes and visibility

```
var wb = new Workbook();
var sheet = wb.AddSheet("View");
var hidden = wb.AddSheet("Lookup");

sheet.View.Freeze(1, 1).ZoomScale = 90;           // freeze row 1 and column A, zoom 90 %
sheet.View.ShowGridLines = false;
sheet.View.Select("B2");
sheet.TabColor = "00B050";
hidden.Visibility = SheetVisibility.Hidden;       // or VeryHidden: only code can unhide it
sheet.Cell("A1").Value = "Visible sheet";
wb.SaveTo("view.xlsx");
```

`SheetView` also has `ShowHeadings`, `ShowZeros`, `ShowFormulas`, `RightToLeft`, `ViewMode` (normal, page break preview, page layout), `TopLeftCell`, split panes (`Split`) and selections. The same `View`, `TabColor` and `Visibility` are on `SheetWriter`. A workbook needs at least one visible sheet, and the active sheet must be visible.

12.2 Outlines and sheet defaults

Rows and columns group with `OutlineLevel` and `Collapsed` (on `sheet.Rows[n]` and `sheet.Columns[n]`, or `RowFormat` and `ColumnFormat` on the streaming writer). `sheet.Format` holds the sheet-wide defaults: default row height and column width, and whether outline summary rows sit below and summary columns to the right.

12.3 Tables

```
using var writer = XlsxWriter.Create("table-stream.xlsx");
using var sheet = writer.AddSheet("Stock");
sheet.WriteHeader("Item", "Qty");
sheet.WriteRow("Gloves", 120);
sheet.WriteRow("Masks", 300);
sheet.AddTable("A1:B3", "StockTable", TableStyle.Medium9, "Item", "Qty");
writer.Save();
```

The table's first row must hold the same header strings that were written. Excel shows filter buttons and the built-in style (`Light1`–`Light21`, `Medium1`–`Medium28`, `Dark1`–`Dark11`). Custom table styles are not supported.

12.4 Hyperlinks and comments

```
var wb = new Workbook();
var sheet = wb.AddSheet("Links");
wb.AddSheet("Other Sheet").Cell("B5").Value = "target";

sheet.Cell("A1").Value = "Website";
sheet.AddHyperlink("A1", target: "https://example.com", tooltip: "Open the site");
sheet.Cell("A2").Value = "Jump";
sheet.AddHyperlink("A2", location: "'Other Sheet'!B5");     // inside the workbook
sheet.Cell("B2").Value = 42;
sheet.AddComment("B2", "Asha", "Checked against the ledger");
wb.SaveTo("links.xlsx");
```

On the streaming writer, `RowBuilder.Hyperlink` and `RowBuilder.Comment` write a linked or annotated cell in one step; `SheetWriter.AddHyperlink(new Hyperlink(...))` and `SheetWriter.AddComment(new CellComment(...))`

attach a link or note to a cell or range written separately. A `CellComment` may carry rich text, a box position (`CommentAnchor`) and `Visible`.

12.5 Protection

Sheet and workbook protection are covered in chapter 24, encryption in chapter 25.

13 Page setup and printing

13.1 Headers and footers

Headers and footers use Excel's codes: `&L`, `&C` and `&R` start the left, centre and right sections; `&P` is the page number, `&N` the page count, `&D` the date, `&T` the time, `&F` the file name and `&A` the sheet name. Text without a section code is centred.

```
var wb = new Workbook();
var sheet = wb.AddSheet("Printed");
sheet.Cell("A1").Value = "Content";
sheet.SetHeaderFooter("&CQuarterly Patient Report", "&LConfidential&RPage &P of &N");
wb.SaveTo("header.xlsx");
```

`PageSetup.HeaderFooter` exposes the full set: odd, even and first-page headers and footers, `DifferentOddEven`, `DifferentFirst`, `ScaleWithDocument` and `AlignWithMargins`.

13.2 Paper, scaling, margins, breaks and print titles

`PageSetup` is the same object on the model (`sheet.PageSetup`), the streaming writer (`SheetWriter.PageSetup`) and the one-line exports (`cfg.PageSetup(p => ...)`):

```
using var writer = XlsxWriter.Create("print.xlsx");
using var sheet = writer.AddSheet("Report");
var page = sheet.PageSetup;
page.PaperSizeCode = 9;                          // A4 (PaperSizeInfo.FromCode(9).Name)
page.Orientation = PageOrientation.Landscape;
page.FitTo(1, 0);                                // one page wide, as many tall as needed
page.Margins.Set(0.5, 0.5, 0.75, 0.75);          // inches: left, right, top, bottom
page.PrintTitleRows = (1, 1);                    // repeat row 1 on every page
page.SetPrintArea("A1:H200");
page.AddRowBreak(50);                            // a new page starts at row 51
page.PrintGridlines = true;
page.CenterHorizontally = true;
sheet.WriteHeader("A", "B", "C");
writer.Save();
```

Setting	Meaning
`PaperSizeCode`, `Paper`	Paper code (9 = A4, 1 = Letter, …); `PaperSizeInfo.All` lists name and size for every code
`Orientation`, `Scale`, `FitTo(wide, tall)`	Orientation; percentage scale, or fit to pages (0 = automatic)
`Margins`	Left, right, top, bottom, header and footer, in inches
`PrintTitleRows`, `PrintTitleColumns`	Rows or columns repeated on every page
`SetPrintArea(...)`, `ClearPrintArea()`	One or more print ranges
`AddRowBreak`, `AddColumnBreak`, `RemoveRowBreak`, …	Manual page breaks
`PrintGridlines`, `PrintHeadings`, `CenterHorizontally`, `CenterVertically`	Print options
`FirstPageNumber`, `PageOrder`, `BlackAndWhite`, `Draft`, `CellComments`, `Errors`, `Copies`	Other printer settings

Nothing is written for settings left at their defaults. On read, the same `PageSetup` comes back from `sheet.PageSetup` or `sheetReader.ReadLayout().PageSetup`, with `PrintAreas` and `PrintTitlesFormula` resolved from the defined names.

14 Data validation

```
var wb = new Workbook();
var sheet = wb.AddSheet("Form");
sheet.Cell("A1").Value = "Status";
sheet.Cell("B1").Value = "Age";
sheet.Cell("C1").Value = "Visit date";

sheet.AddDataValidation("A2:A100", DataValidation.List("Open", "Closed", "On hold"));

var age = DataValidation.WholeNumber(0, 120);
age.ErrorTitle = "Age";
age.ErrorMessage = "Enter a whole number from 0 to 120.";
age.PromptMessage = "Age in years";
sheet.AddDataValidation("B2:B100", age);

sheet.AddDataValidation("C2:C100",
    DataValidation.Date(new DateTime(2026, 1, 1), new DateTime(2026, 12, 31)));
wb.SaveTo("validation.xlsx");
```

Factory	Accepts
`List(values)`	One of the given values (a drop-down)
`ListFromRange("Lists!$A$1:$A$10")`	One of the values in a range or named range
`WholeNumber(min, max)`, `DecimalNumber(min, max)`	A number in range (either bound may be `null`)
`Date(min, max)`, `Time(min, max)`	A date or time in range
`TextLength(min, max)`	Text of a length in range
`Custom(formula)`	Anything the formula accepts
`Create(kind, comparison, f1, f2)`	The full option set: any type with any comparison

Each rule has `AllowBlank`, `ShowDropdown`, `ErrorStyle` (stop, warning, information), `ErrorTitle`, `ErrorMessage`, `PromptTitle` and `PromptMessage`. On read, `sheet.DataValidations` lists every rule with its range.

15 Conditional formatting

15.1 Writing rules

```
var wb = new Workbook();
var sheet = wb.AddSheet("Scores");
sheet.Cell("A1").Value = "Score";
for (int r = 2; r <= 11; r++) sheet.Cell(r, 1).Value = (r - 1) * 10;
for (int r = 2; r <= 11; r++) sheet.Cell(r, 2).Value = (r - 1) * 7;

var red = CellStyle.Default with { FontColor = "9C0006", FillColor = "FFC7CE" };
sheet.AddConditionalFormat("A2:A11", ConditionalFormat.CellIs(ComparisonOp.LessThan, 30, red));
sheet.AddConditionalFormat("B2:B11", ConditionalFormat.ColorScale.RedWhiteGreen);
sheet.AddConditionalFormat("C2:C11", ConditionalFormat.DataBar.Blue);
sheet.AddConditionalFormat("A2:B11", ConditionalFormat.Expression("MOD(ROW(),2)=0",
    new DifferentialStyle { FillColor = "F2F2F2" }));
wb.SaveTo("conditional.xlsx");
```

Helper	Rule
`ConditionalFormat.CellIs(op, value or formula, style)`, `Between(min, max, style)`	Compare the cell's value
`ConditionalFormat.Expression(formula, style)`	Any formula, relative to the range's top-left cell
`ConditionalFormat.ColorScale.RedWhiteGreen`, `GreenWhiteRed`, `BlueWhiteRed`, `Custom(...)`	Two- or three-colour scales
`ConditionalFormat.DataBar.Blue`, `Green`, `Red`, `Custom(colour)`	Data bars

The style of a comparison or expression rule is written as a differential format: only what it sets (font colour, bold, italic, underline, strikethrough, fill, number format, borders) overrides the cell's own style.

15.2 The full rule set

For every other rule type, build a `ConditionalRule` directly and add it in a `ConditionalFormatting` block: top/bottom N, above or below average, duplicate and unique values, text contains, begins or ends with, blanks, errors, time periods, icon sets, and data bars with the Excel 2010 details (gradient, borders, negative colours, axis, direction).

```
var wb = new Workbook();
var sheet = wb.AddSheet("Rules");
for (int r = 1; r <= 20; r++) sheet.Cell(r, 1).Value = r * 3 % 17;

var top3 = new ConditionalRule(ConditionalRuleType.Top10)
{
    Rank = 3,
    Style = new DifferentialStyle { Bold = true, FillColor = "C6EFCE" },
};
var dupes = new ConditionalRule(ConditionalRuleType.DuplicateValues)
{
    Style = new DifferentialStyle { FontColor = "9C0006" },
};
var icons = new ConditionalRule(ConditionalRuleType.IconSet)
{
    IconSet = new IconSetInfo("3Arrows", new[]
    {
        new ConditionalValue(ConditionalValueType.Percent, "0"),
        new ConditionalValue(ConditionalValueType.Percent, "33"),
        new ConditionalValue(ConditionalValueType.Percent, "67"),
```

```
    }),
};
sheet.AddConditionalFormatting(
    new ConditionalFormatting(new[] { "A1:A20" }, new[] { top3, dupes, icons }));
wb.SaveTo("rules.xlsx");
```

Rules are applied in priority order; a rule with `StopIfTrue` stops lower-priority rules for the cells it matches. Rules loaded from a file keep their priorities and round-trip on load and save. Chapter 21 shows how to evaluate them per cell, as Excel shows them.

16 The streaming reader

16.1 Opening a file

```
var wb = new Workbook();
var s = wb.AddSheet("Staff");
s.Cell("A1").Value = "Id";  s.Cell("B1").Value = "Name";      s.Cell("C1").Value = "Joined";
s.Cell("A2").Value = 7;     s.Cell("B2").Value = "Asha Rao";
s.Cell("C2").Value = new DateTime(2019, 6, 1);
s.Cell("A3").Value = 9;     s.Cell("B3").Value = "Ravi Iyer";
s.Cell("C3").Value = new DateTime(2021, 2, 15);
wb.AddSheet("Notes").Cell("A1").Value = "none";
wb.SaveTo("staff.xlsx");

using var reader = XlsxReader.Open("staff.xlsx");
foreach (var info in reader.Sheets)
    Console.WriteLine($"{info.WorkbookIndex}: {info.Name} ({info.Visibility})");

var staff = reader.Sheet("Staff");               // or reader.Sheet(1): 1-based
foreach (var row in staff.Rows)
    Console.WriteLine($"row {row.RowNumber}: {row.GetInt32("Id")} {row.GetString("Name")} "
                      + $"{row.GetDateTime("Joined"):yyyy-MM-dd}");
```

- `XlsxReader.Open` takes a path or a stream (left open), optional `XlsxReaderOptions`, and for encrypted files a password (chapter 25).
- `reader.Sheets` lists the worksheets in tab order before any sheet is read; `ChartSheets` lists chart sheets.
- `Sheet(name)` or `Sheet(index)` returns a `SheetReader`; enumerating `Rows` streams the sheet's XML once.
- The reader keeps the package open until it is disposed.

16.2 Rows

Each row is a `RowReader`, a transient view over a buffer that the next row reuses. Copy out what you need inside the loop; do not keep the `RowReader` itself.

Member	Gives
`RowNumber`	The 1-based row number in the sheet (gaps are skipped, not filled)
`Count`	Number of cells: the highest column present
`row[i]`, `row["Name"]`	The raw value by 0-based column or by header text, or `null`
`GetString`, `GetInt32`, `GetInt64`, `GetDouble`, `GetDecimal`, `GetBool`, `GetDateTime`, `Get<T>`	Typed values; throw `InvalidOperationException` for a missing cell and `InvalidCastException` for a value of another type
`TryGet<T>(col, out value)`	`false` instead of an exception
`HasCell(i)`	Whether the file has a cell element there (a styled empty cell counts)
`GetDisplayString(col)`	The text as Excel shows it (chapter 19)
`GetFormula(col)`, `GetFormulaInfo(col)`, `HasCachedValue(col)`	The formula and whether a result is stored

Member	Gives
`GetRichText(col)`	The runs of a rich-text string (chapter 22)
`GetStyle(col)`, `GetStyleIndex(col)`, `GetEffectiveStyleIndex(col, layout)`	Styles (chapter 20)
`Attributes`	Row height, hidden, outline level, row style

```
var noHeaders = new XlsxReaderOptions { TreatFirstRowAsHeaders = false };
using var reader = XlsxReader.Open("staff.xlsx", noHeaders);
var copied = new List<(int Id, string Name)>();
foreach (var row in reader.Sheet(1).Rows)
{
    if (row.RowNumber == 1) continue;             // the header row is data when headers are off
    if (row.TryGet<double>(0, out var id) && row[1] is string name)
        copied.Add(((int)id, name));
}
Console.WriteLine(string.Join(", ", copied));
```

16.3 Header names

With `TreatFirstRowAsHeaders` (the default) the first row is read as headers: it is not returned, and its text addresses columns in `row["Name"]`, `TryGet("Name", ...)`, `GetString("Name")` and the other named overloads.

- Only text cells in row 1 become headers, exactly as written: spaces are not trimmed, and numbers or dates in row 1 give no name.
- Names match case-insensitively (ordinal, ignoring case): `row["name"]` and `row["NAME"]` find the column headed `Name`. When two headers differ only in case or repeat, the first (leftmost) one wins.
- A name that is not a header reads like an empty cell: `row["Nmae"]` is `null`, `TryGet` returns `false`, `GetDisplayString` returns `""`, `GetFormula` and `GetRichText` return `null`, and the typed getters (`GetString`, `GetInt32`, `GetInt64`, `GetDouble`, `GetDecimal`, `GetBool`, `Get<T>`) throw `InvalidOperationException` ("Cell 'Nmae' is null or missing."), except `GetDateTime`, which throws `InvalidCastException` ("Cell 'Nmae' is not a date."). No exception names the unknown header, so check names against the header row when they come from outside.
- With headers off, every row is returned and only 0-based indexes work: any name throws `InvalidOperationException` ("Header index not built; enable TreatFirstRowAsHeaders.").

16.4 Options that change what is read

Option	Default	Effect
`TreatFirstRowAsHeaders`	`true`	As above
`AutoDetectDates`	`true`	Numbers with a date or time format are returned as `DateTime`
`StrictXml`	`false`	With `false`, a malformed theme or chart part is skipped; with `true`, any malformed part throws `XlsxFormatException`
`IncludeStyledEmptyCells`	`false`	For `Workbook.Load`: also load empty cells that carry a style (for rendering)
`LoadImageData`	`true`	For `Workbook.Load`: read picture bytes into memory
`DisplayCulture`	invariant	Separators, month and day names for display text
Limits and `CancellationToken`	off	Chapter 26

17 Loading a workbook

`Workbook.Load` reads a whole file into the model: values, formulas and cached results, styles, merges, row and column settings, views, page setup, defined names, conditional formats, validation, rich text, hyperlinks, comments, drawings, charts and header/footer pictures.

```
var wb = Workbook.Load("staff.xlsx");
foreach (var sheet in wb.Sheets)
{
    Console.WriteLine($"{sheet.Name}: {sheet.PopulatedCells.Count()} cells");
    foreach (var cell in sheet.PopulatedCells.Take(3))
        Console.WriteLine($"  {cell.Address} = {cell.Value} shown as '{cell.DisplayText}'");
}
Console.WriteLine($"macros: {wb.HasMacros}, strict: {wb.IsStrict}, 1904 dates: {wb.Date1904}");
```

Overload	Use
`Load(path)`, `Load(stream)`	Plain files
`Load(path, password)`	Possibly encrypted files: the password is ignored for an unencrypted file
`Load(path, password, options)`, `Load(stream, options)`, `Load(stream, password, options)`	With reader options: limits, cancellation, culture, styled empty cells

`Workbook.Load` always reads row 1 as data; `TreatFirstRowAsHeaders` does not apply to the model.

What the model could not hold is reported, never dropped silently: `wb.LoadWarnings` lists, for example, cells outside the 1,048,576 × 16,384 grid that some third-party writers produce. The streaming reader reports the same as `SheetReader.OutOfGridCells`, `OutOfGridRows` and `FirstOutOfGridReference`.

For read-only access to large files, prefer `XlsxReader`: the model holds every populated cell. A 10 MB workbook of 1.75 million cells loads in about 1.4 to 2 s and keeps about 127 bytes per cell (chapter 28).

18 Typed and table reads

18.1 Into objects

`ReadXlsx<T>` maps the header row to public settable properties by header text, case-insensitively. A property's `[Column("...")]` header is used when it has one, so a list exported with `ToXlsx` reads back into the same type. `[ColumnIgnore]` properties are skipped.

```
var patients = new[]
{
    new Patient { Id = "P-001", Name = "Asha Rao",
                  DateOfBirth = new DateTime(1980, 1, 5), Ward = "A" },
};
patients.ToXlsx("roundtrip.xlsx");

List<Patient> back = XlsxReadExtensions.ReadXlsx<Patient>("roundtrip.xlsx");
Console.WriteLine($"{back[0].Id} {back[0].Name} {back[0].DateOfBirth:yyyy-MM-dd}");
```

Values are converted to the property type: numbers to any numeric type, numbers to `bool`, date serials to `DateTime`, everything else through `Convert.ChangeType` with the invariant culture.

18.2 Into DataTables

```
DataSet all = XlsxReadExtensions.ReadXlsx("staff.xlsx");              // one table per sheet
DataTable first = XlsxReadExtensions.ReadXlsxSheet("staff.xlsx");     // the first sheet
DataTable named = XlsxReadExtensions.ReadXlsxSheet("staff.xlsx", "Staff");
Console.WriteLine($"{all.Tables.Count} tables; {named.Rows.Count} rows in Staff");
```

The first row is always the header row. Every overload has a form with a password and `XlsxReaderOptions`, so the one-liners can open encrypted and untrusted files too.

19 Display text and number formats

19.1 What a person sees

A cell's value and its displayed text differ: 45000 under `dd/mm/yyyy` shows as `15/03/2023`. Chuvadi.Sheets renders values with Excel's number format rules: sections for positive, negative, zero and text, colours, conditions, thousands and scaling commas, percentages, fractions, scientific notation, elapsed times, and date and time codes.

```
Console.WriteLine(NumberFormatter.Format(45000, "dd/mm/yyyy"));          // 15/03/2023
Console.WriteLine(NumberFormatter.Format(1234.5, "#,##0.00"));           // 1,234.50
Console.WriteLine(NumberFormatter.Format(0.256, "0.0%"));                // 25.6%
Console.WriteLine(NumberFormatter.Format(-5, "#,##0;[Red](#,##0)"));    // (5)
Console.WriteLine(NumberFormatter.Format(1.5, "[h]:mm"));                // 36:00

var acct = NumberFormatter.FormatDetailed(1234.5, "_($* #,##0.00_)");
Console.WriteLine($"'{acct.Text}', fill '{acct.FillCharacter}' at {acct.FillIndex}");
// ' $1,234.50 ', fill ' ' at 2

var red = NumberFormatter.FormatDetailed(-5, "#,##0;[Red]-#,##0");
Console.WriteLine($"{red.Text} in {red.Color}");                         // -5 in FF0000
```

`FormatDetailed` also returns what a renderer needs beyond the text: the colour from a `[Red]`-style code, the repeat-fill character from a `*` code and where it goes, and whether a date or time section was used.

19.2 On read

- `row.GetDisplayString(col)` and `cell.DisplayText` render the value with the cell's own number format, the file's date system and the culture in `DisplayCulture`.
- Formula cells show their cached result; a formula without one shows empty text.
- Culture-dependent built-in formats (currency and accounting ids 5 to 8 and 41 to 44) render as Excel does in en-US.

```
var wb = new Workbook();
var sheet = wb.AddSheet("Money");
sheet.Cell("A1").Value = 1234567.891;
sheet.Cell("A1").Style = CellStyle.Default with { NumberFormat = "#,##0.00" };
wb.SaveTo("money.xlsx");

var options = new XlsxReaderOptions
{
    TreatFirstRowAsHeaders = false,
    DisplayCulture = new CultureInfo("de-DE"),
};
using var reader = XlsxReader.Open("money.xlsx", options);
foreach (var row in reader.Sheet(1).Rows)
    Console.WriteLine(row.GetDisplayString(0));                          // 1.234.567,89
```

`NumberFormatOptions` (`Culture`, `Date1904`) gives the same control to `NumberFormatter` and `cell.GetDisplayText(options)`.

20 Styles on read

20.1 Three layers

A loaded cell's style can be read at three levels of detail:

1. **Resolved:** `cell.Style` (or `row.GetStyle(col)`) is a flat `CellStyle`: font, fill, borders, number format, alignment and protection, with theme and tint colours resolved to `RRGGBB`. It is `null` for cells that use the workbook default format; `StyleTable.DefaultCellStyle` is that default.
2. **Raw index:** `cell.StyleIndex` (or `row.GetStyleIndex(col)`) is the cell's `s=` attribute, an index into `StyleTable.CellFormats`; `-1` when the file gave none.
3. **The full table:** `wb.StyleTable` (or `reader.StyleTable`), a `CellStyleTable`, holds every list from `styles.xml` and the theme, exactly as stored: fonts, fills, borders, number formats, cell and named-style formats, named styles, differential formats, the indexed palette and the theme's colour and font schemes.

```
var options = new XlsxReaderOptions { IncludeStyledEmptyCells = true };
var book = Workbook.Load("styles.xlsx", null, options);
var sheet = book.Sheets[0];
var cell = sheet.Cell("A1");

CellStyle effective = cell.Style ?? book.StyleTable.DefaultCellStyle ?? CellStyle.Default;
Console.WriteLine($"bold {effective.Bold}, fill {effective.FillColor}, font {effective.FontColor}");

int xf = cell.StyleIndex < 0 ? 0 : cell.StyleIndex;
var format = book.StyleTable.CellFormats[xf];
var font = book.StyleTable.Fonts[format.FontId];
Console.WriteLine($"{font.Name} {font.Size} colour {book.StyleTable.ResolveColor(font.Color)}");
Console.WriteLine($"number format: {book.StyleTable.GetNumberFormatCode(format.NumberFormatId)}");
```

`ColorRef` keeps a colour as stored (RGB, theme slot with tint, indexed, or automatic); `ResolveColor` turns it into `RRGGBB` with the workbook's theme and palette.

20.2 What round-trips

On load and save, these style properties are kept: fonts, colours, solid fills, all border styles, number formats with a code, horizontal and vertical alignment, and wrap. Merges and column widths are kept too.

These are read but not written back: underline kind, strikethrough, superscript and subscript, pattern and gradient fills, diagonal borders, indent, rotation, shrink-to-fit, reading order, cell protection, quote prefix, and culture-dependent built-in number formats (for example id 44). Differential formats are read through `CellStyleTable.DifferentialFormats`; conditional-format and validation rules round-trip with their own formats.

21 Layout for renderers

21.1 The layout of a streamed sheet

`SheetReader.ReadLayout()` returns everything a renderer or paginator needs before reading rows: the page setup with print areas and titles, sheet format defaults, column spans, merged ranges, the used range, the view, hyperlinks, conditional formats and validation. The data sits in the middle of the sheet part; `ReadLayout` makes one pass that skips the cell data without parsing it, and caches the result. Call it before or between row enumerations. `UsedRange` is the range the file states in its `<dimension>` element, or `null` when it states none, as Chuvadi-written files do.

```
var noHeaders = new XlsxReaderOptions { TreatFirstRowAsHeaders = false };
using var reader = XlsxReader.Open("ranges.xlsx", noHeaders);
var sheet = reader.Sheet(1);
SheetLayoutInfo layout = sheet.ReadLayout();
Console.WriteLine($"merges {string.Join(" ", layout.MergeRanges)}");
Console.WriteLine($"A width {layout.ColumnWidth(1):0.##}, D hidden {layout.IsColumnHidden(4)}");
Console.WriteLine($"frozen rows {layout.View.FrozenRows}, tab colour {layout.TabColor}");

foreach (var row in sheet.Rows)
{
    double height = row.Attributes.Height;
    for (int c = 0; c < row.Count; c++)
    {
        // the cell's own style, else the row style, else the column style
        int xf = row.GetEffectiveStyleIndex(c, layout);
    }
}
```

`Sheet.GetEffectiveStyle(row, column)` gives the same for the model without creating cells.

21.2 Conditional formats as Excel shows them

```
var wb = Workbook.Load("conditional.xlsx");
var sheet = wb["Scores"];
var cf = sheet.EvaluateConditionalFormats();
for (int r = 2; r <= 5; r++)
{
    var result = cf.Evaluate(r, 1);                  // null when no rule covers the cell
    var own = sheet.FindCell(r, 1)?.Style ?? CellStyle.Default;
    var shown = result?.ApplyTo(own) ?? own;         // fill, font, number format, borders
    Console.WriteLine($"A{r}: fill {shown.FillColor ?? "none"}, font {shown.FontColor ?? "auto"}");
}
var bar = cf.Evaluate(2, 3)?.DataBar;                // a data bar to draw (length, colours)
```

The evaluator applies rules in priority order, as Excel does: a property set by a higher-priority rule wins, and `StopIfTrue` stops lower rules. It evaluates cell-value comparisons, expressions of common functions, colour scales, data bars, icon sets, top/bottom, averages, duplicates, text, blanks, errors and time periods (`ConditionalFormatOptions.Today` fixes "today" for tests). Rules it cannot evaluate, such as formulas that refer to other sheets or use unsupported functions, are listed in `result.NotEvaluated` so a renderer can say so.

For the streaming reader, construct a `ConditionalFormatEvaluator` from `layout.ConditionalFormats`, `reader.StyleTable` and your own `ICellValueSource` over the rows you hold.

22 Rich text, hyperlinks and comments

22.1 Rich text

```
var title = new RichText(
    new RichTextRun("Total: ", new RichTextFont { Bold = true }),
    new RichTextRun("42", new RichTextFont { Color = ColorRef.FromRgb("FF00B050") }));

var wb = new Workbook();
var sheet = wb.AddSheet("Rich");
sheet.Cell("A1").RichText = title;              // Value becomes "Total: 42"
wb.SaveTo("rich.xlsx");

var loaded = Workbook.Load("rich.xlsx")["Rich"].FindCell(1, 1);
foreach (var run in loaded?.RichText?.Runs ?? Array.Empty<RichTextRun>())
    Console.WriteLine($"'{run.Text}': bold={run.Font?.Bold} colour={run.ResolvedColor}");
```

A rich string is written as runs, each with its own `RichTextFont` (name, size, bold, italic, underline, strikethrough, superscript and subscript, colour). On the streaming writer, pass a `RichText` as a cell value; on read, `row.GetRichText(col)` returns the runs and `row[col]` the plain text. Run colours are resolved against the theme in `ResolvedColor`.

22.2 Hyperlinks

```
var links = Workbook.Load("links.xlsx")["Links"];
foreach (var link in links.Hyperlinks)
    Console.WriteLine($"{link.Reference}: {(link.IsInternal ? link.Location : link.Target)}");
Console.WriteLine(links.FindHyperlink(1, 1)?.Target);
```

A `Hyperlink` points outside the workbook (`Target`: a URL, `mailto:` or file) or inside it (`Location`, such as `'Other Sheet'!B5`), covers a cell or a range, and may have a tooltip and display text. The streaming reader gives the same list in `layout.Hyperlinks`.

22.3 Comments

`sheet.Comments` (and `sheetReader.ReadComments()`) list the notes with their author, text and runs, box anchor and visibility. Comments round-trip on load and save. Threaded comments of Excel 365 are read through the note Excel writes alongside them.

23 Drawings, charts and header pictures

Pictures, shapes, connectors, groups and charts are read, with their anchors, but not written.

```
void ExtractPictures(string path, string folder)
{
    var wb = Workbook.Load(path);
    foreach (var sheet in wb.Sheets)
    foreach (var d in sheet.Drawings)
    {
        var from = d.Anchor.From;                    // cell corner plus offset, 1-based
        Console.WriteLine($"{sheet.Name}: {d.Kind} '{d.Name}' at C{from?.Column} R{from?.Row}");
        if (d.Kind == DrawingKind.Picture && d.GetImageBytes() is byte[] bytes)
            File.WriteAllBytes(Path.Combine(folder, $"{sheet.Name}-{d.Id}.img"), bytes);
        if (d.Kind == DrawingKind.Chart && d.Chart is ChartInfo chart)
            foreach (var series in chart.AllSeries)
                Console.WriteLine($"  {chart.Title}: {series.Name} = {series.Values?.Formula}");
    }
}
```

- `DrawingObject` has the kind, anchor (`TwoCell`, `OneCell` or `Absolute`, in EMU; `Emu` converts to points, inches and pixels), name, description, rotation, flips, group children, picture crop and content type, shape geometry, text, fill and line, and for charts a `ChartInfo`.
- `ChartInfo` gives the chart type, title, plots and series with their data references and the values cached in the chart part, axis titles and legend position. Excel 2016 chart types are reported as `Extended`.
- `sheet.HeaderFooterPictures` lists pictures in headers and footers (the `&G` code), with section and size.
- `wb.ChartSheets` (and `reader.ReadChartSheet(info)`) give chart sheets: tabs that hold one chart.
- With `XlsxReaderOptions.LoadImageData = false`, `Workbook.Load` keeps anchors and metadata only. The streaming reader (`sheetReader.ReadDrawings()`) always reads picture bytes on demand, while the reader is open.

24 Sheet and workbook protection

Protection is Excel's cooperative edit lock. It stops casual changes in Excel, but it is not encryption: the content stays readable, and a determined user can remove it. For confidentiality, encrypt the workbook (chapter 25).

24.1 Sheet protection

```
var wb = new Workbook();
var sheet = wb.AddSheet("Locked");
sheet.Cell("A1").Value = "Read me, do not edit me";
sheet.Protect("sheet-password", new SheetProtectionOptions
{
    AllowSelectLockedCells = true,
    AllowSort = true,
    AllowAutoFilter = true,
});
wb.SaveTo("protected.xlsx");
```

The password is not stored. It is hashed with SHA-512, a random salt and 100,000 iterations, as Excel does, and Excel asks for the same password to unprotect. `SheetProtectionOptions` defaults match Excel's "Protect Sheet" dialog: selecting locked and unlocked cells is allowed, everything else is not. Each `Allow...` flag permits one action: formatting cells, columns or rows; inserting columns, rows or hyperlinks; deleting columns or rows; sorting; autofilter; pivot tables; objects; scenarios.

Every cell is locked while its sheet is protected, which is Excel's default. In 2.0.0 the writer does not write a per-cell "unlocked" flag: `CellStyle.Locked` is filled in on read but not written (chapter 20).

The streaming writer has the same method: `SheetWriter.Protect(password, options)`.

24.2 Workbook structure protection

```
var wb = new Workbook();
wb.AddSheet("One").Cell("A1").Value = 1;
wb.AddSheet("Two").Cell("A1").Value = 2;
wb.Protect("structure-password", lockStructure: true, lockWindows: false);
wb.SaveTo("structure.xlsx");
```

Structure protection stops sheets being added, deleted, renamed, moved, hidden or unhidden. It is independent of sheet protection. On the streaming writer: `writer.ProtectWorkbook(password, lockStructure, lockWindows)`.

25 Encryption

25.1 Writing an encrypted workbook

```
var wb = new Workbook();
wb.AddSheet("Private").Cell("A1").Value = "Confidential";
wb.SaveTo("encrypted.xlsx", new EncryptionOptions { Password = "open-sesame" });

using (var writer = XlsxWriter.Create("encrypted-stream.xlsx", new XlsxWriterOptions
{
    Encryption = new EncryptionOptions { Password = "open-sesame" },
}))
{
    using var sheet = writer.AddSheet("Data");
    for (int i = 1; i <= 10_000; i++) sheet.WriteRow(i, "row " + i);
    writer.Save();
}
```

Excel asks for the password when it opens either file. The encryption is the format Excel itself uses:

- OOXML Agile Encryption, [MS-OFFCRYPTO] sections 2.3.4.10 to 2.3.4.15;
- AES-256-CBC for the content, an iterated SHA-512 key derivation, and an HMAC-SHA512 integrity check that is verified on read: a tampered or damaged file fails instead of decrypting to garbage;
- a Compound File Binary container, as Excel writes; above about 7 MB it is extended with DIFAT sectors ([MS-CFB] section 2.5), as Excel does.

`EncryptionOptions.SpinCount` sets the key-derivation iterations: 100,000 by default, as modern Excel writes. Higher is slower to attack and slower to open; do not go below 50,000.

The streaming writer encrypts in 4,096-byte segments with an incrementally computed HMAC, so a large export is never held in memory as plaintext. Its temporary files are encrypted with a one-time key held only in memory (section 7.6). A save to a stream writes from the stream's current position and works on a non-seekable stream.

Before 2.0.1, a file that needed more than 109 FAT sectors (an encrypted file larger than about 7 MB) was saved without an error but could not be opened by any reader. 2.0.1 writes such files correctly, and files below that size are written byte for byte as before. Files saved by an earlier version cannot be repaired: save them again.

Round trips are verified in the library and against an independent implementation of MS-OFFCRYPTO. Opening a sample file in desktop Excel remains a sensible final check for your own deployment.

25.2 Reading an encrypted workbook

```
var wb = Workbook.Load("encrypted.xlsx", "open-sesame");
Console.WriteLine(wb["Private"].Cell("A1").Value);

using var reader = XlsxReader.Open("encrypted-stream.xlsx", "open-sesame",
                                   new XlsxReaderOptions());
Console.WriteLine(reader.Sheet(1).Rows.Count());

try
{
    Workbook.Load("encrypted.xlsx", "wrong");
}
catch (XlsxPasswordRequiredException ex)
{
    Console.WriteLine("Password problem: " + ex.Message);               // Incorrect password.
}
```

- A missing password and a wrong password both throw `XlsxPasswordRequiredException`.
- For an unencrypted file the password is ignored, so one code path opens both.
- Decryption happens in memory.
- With a password given, a damaged container or a failed integrity check throws `XlsxFormatException`. Without a password the container is not read, so damage is found only once a password is given.

25.3 Spin counts and antivirus

The key-derivation cost is declared by the file, so a hostile file could demand billions of hash rounds. Reading caps it with `XlsxReaderOptions.MaxSpinCount` (default 500,000; exceeding it throws `XlsxLimitExceededException` before any derivation), and the reader's `CancellationToken` interrupts derivation every 4,096 rounds.

On Windows, real-time antivirus (Microsoft Defender) scans an encrypted Office file when it is opened, by trying Excel's default password, which runs the full key derivation at the file's spin count. That happens inside the operating system's file-open call, before the library reads a byte, so neither `MaxSpinCount` nor the token can shorten it. A 3,000,000-spin file took about 13 s to open the first time on a test machine and 9 ms once the scan result was cached. For uploads, read the bytes into memory and open a `MemoryStream`: the limits then apply in full.

26 Untrusted input

26.1 Limits

A few kilobytes of xlsx can legally inflate to gigabytes, declare a billion rows, or demand an expensive key derivation. For files you did not create, set limits:

```
void HandleUpload(Stream upload, string? password, CancellationToken requestAborted)
{
    var options = XlsxReaderOptions.ForUntrustedInput();
    options.CancellationToken = requestAborted;
    options.TreatFirstRowAsHeaders = false;

    try
    {
        using var reader = XlsxReader.Open(upload, password, options);
        foreach (var row in reader.Sheet(1).Rows)
            Console.WriteLine(row.GetDisplayString(0));
    }
    catch (XlsxFormatException ex)
    {
        Console.WriteLine("Not a valid workbook: " + ex.Message);
    }
    catch (XlsxPasswordRequiredException)
    {
        Console.WriteLine("Password missing or wrong.");
    }
    catch (XlsxLimitExceededException ex)
    {
        Console.WriteLine($"Too large: {ex.LimitName} {ex.Actual} > {ex.Limit}");
    }
    catch (InvalidDataException ex) when (ex.Data.Contains("Chuvadi.Sheets.ReadLimit"))
    {
        Console.WriteLine("A part is too large: " + ex.Message);
    }
}
```

Limit	`ForUntrustedInput()`	On excess
`MaxPartBytes`: decompressed size of one part	256 MB	`InvalidDataException` (see 26.2)
`MaxTotalUncompressedBytes`: decompressed size of the whole package, and the size of an encrypted file	1 GB	`XlsxLimitExceededException`
`MaxCompressionRatio`: per entry larger than 1 MB	500:1	`XlsxLimitExceededException`
`MaxPackageEntries`: zip entries	20,000	`XlsxLimitExceededException`
`MaxRowsPerSheet`: rows read from one sheet	1,048,576 (Excel's maximum)	`XlsxLimitExceededException`
`MaxSpinCount`: key-derivation iterations of an encrypted file	500,000 (also the plain default)	`XlsxLimitExceededException`

Without `ForUntrustedInput()` the limits are off (null), except `MaxSpinCount`, for compatibility with trusted input. Sizes are checked twice: against what the zip declares, before anything is read, and against the bytes actually inflated while reading, because declared sizes can lie. `XlsxLimitExceededException` derives from `IOException` and names the limit in `LimitName`, with `Limit` and `Actual`.

The `CancellationToken` is checked while the package is opened, inside key derivation, and on every row, so a cancel takes effect well within a second as `OperationCanceledException`.

26.2 The `MaxPartBytes` marker

`MaxPartBytes` has thrown `InvalidDataException` since before the other limits existed, and keeps doing so with the same message: `Decompressed size limit of {n} bytes exceeded for package part '{name}' (declared {m}).` when the zip declares a part larger than the limit, and `Decompressed size limit of {n} bytes exceeded for package part '{name}'.` when more bytes than that are inflated from a part. It is not wrapped in `XlsxFormatException`.

Since 2.0.0 that exception carries `Data["Chuvadi.Sheets.ReadLimit"] = true` (a boxed `bool`), set at both places the limit is checked. It is how the library, and your code, tell this size limit apart from an `InvalidDataException` caused by a damaged zip, which 2.0.0 reports as `XlsxFormatException`.

26.3 Macros

`reader.HasMacros` and `wb.HasMacros` report a VBA project or Excel 4.0 macro sheets. Macros are never run, and saving a loaded workbook writes a macro-free `.xlsx`.

27 Errors

27.1 What the readers throw

Exception	When
`XlsxFormatException`	The file is malformed: not a zip or a damaged zip, malformed XML in any part, a missing required part, a damaged relationship entry, a truncated part, or, with a password given, a damaged encrypted container or a failed integrity check. `InnerException` holds the original error.
`XlsxPasswordRequiredException`	The file is encrypted and no password was given, or the password is wrong.
`XlsxLimitExceededException` (an `IOException`)	A read limit was exceeded: `MaxRowsPerSheet`, `MaxTotalUncompressedBytes`, `MaxCompressionRatio`, `MaxPackageEntries`, `MaxSpinCount`.
`InvalidDataException` with `Data["Chuvadi.Sheets.ReadLimit"]`	`MaxPartBytes` was exceeded (a limit, not a malformed file).
`OperationCanceledException`	The `CancellationToken` was cancelled.
`ArgumentException`, `ArgumentNullException`, `KeyNotFoundException`, `InvalidCastException`, `InvalidOperationException`	Caller mistakes: a bad argument, no sheet with that name, a typed getter on a value of another type, a call in the wrong state.
I/O exceptions (`FileNotFoundException`, `UnauthorizedAccessException`, …)	Opening a path failed.

This applies to `XlsxReader.Open`, `Workbook.Load`, row enumeration, `ReadLayout`, `ReadDrawings`, `ReadComments`, `ReadHeaderFooterPictures`, `ReadChartSheet`, `GetImageBytes` and the `ReadXlsx` extensions. With `StrictXml` off (the default), a malformed theme or chart part is skipped rather than reported.

```
File.WriteAllText("not-a-workbook.xlsx", "plain text, not a zip");
try
{
    Workbook.Load("not-a-workbook.xlsx");
}
catch (XlsxFormatException ex)
{
    Console.WriteLine($"{ex.GetType().Name}, inner {ex.InnerException?.GetType().Name}");
}
```

27.2 Writers

The writers throw `ArgumentException` and its relatives for invalid input, such as an empty password or a page setting out of range, and `InvalidOperationException` for a call in the wrong state, such as `AutoFitColumns` after rows were written, or a save with no visible sheet or with a hidden active sheet.

28 Performance

28.1 Measured numbers

Measured with the library's own `perf` group, which generates a 10 MB workbook (3 sheets × 52,000 rows × 12 columns: ids, dates, styled money, repeated categories, status, sparse remarks; 1.75 million cells) and reads it from memory:

Measure	2 cores, Windows	2 cores, Linux
`Workbook.Load`, warm median	1.2 s	1.4 to 1.8 s
`XlsxReader`, every cell, warm median	0.5 s	0.5 to 0.7 s
Memory kept by the loaded model	127 bytes per cell (213 MB)	the same
Peak working set of `Workbook.Load`	not measured	about 0.66 GB
Saving 50,000 hyperlinks	0.4 s	0.2 to 0.4 s

The first load in a process is slower, by roughly a second, because of JIT compilation; a ReadyToRun build of the application shortens it. Run the group yourself:

```
dotnet run -c Release --project samples\Chuvadi.Sheets.ManualTests -- perf
```

28.2 Choosing for speed and memory

- **Writing many rows:** `XlsxWriter`. Memory is flat; the cost is one temporary file per sheet.
- **Reading many rows:** `XlsxReader`. One reused buffer per row, about 250 MB peak for 1.75 million cells, no model built.
- **Display text** costs a little on top of reading values (about 0.3 s for 1.75 million cells).
- **`Workbook.Load`** holds every populated cell; use it when you need the whole model.
- **Repeated strings** are written once through the shared strings table; `UseInlineStrings` trades file size for a simpler save.
- **Open from memory** when the file arrived over the network: on Windows, antivirus scans files on open, and an encrypted file's scan can take seconds (chapter 25).

29 Threading

No public type is thread-safe for concurrent use. Use each instance from one thread at a time; separate instances are independent.

Operation	Guarantee
Two threads writing to the same `XlsxWriter`, or to different `SheetWriter`s of it	Not supported: sheet writers share the parent's state
Two threads each with their own `XlsxWriter` or `XlsxReader`	Safe
Two threads enumerating sheets of the same `XlsxReader`	Not supported
Two threads each with their own `XlsxReader` over the same file	Safe: each opens its own stream
Sharing a `CellStyle` between threads	Safe: it is an immutable value
`NumberFormatter`	Safe: parsed format codes are cached thread-safely

The asynchronous methods (`WriteRowAsync`, `SaveAsync`, `DisposeAsync`) make I/O non-blocking but add no concurrency: one call at a time per writer. For a pipeline, feed one writer from a queue:

```
async Task ExportAsync(IAsyncEnumerable<(int Id, string Name)> source, string path)
{
    await using var writer = XlsxWriter.Create(path);
    await using (var sheet = writer.AddSheet("Data"))
    {
        await sheet.WriteHeaderAsync("Id", "Name");
        await foreach (var (id, name) in source)
            await sheet.WriteRowAsync(id, name);
    }
    await writer.SaveAsync();
}
```

30 The zip API (obsolete)

`Chuvadi.Sheets.Zip` (`ZipWriter`, `ZipReader`, `ZipEntryInfo`, `ZipExtractionLimits` and the zip extension methods) is obsolete since 2.0.0, and `ZipFormatException` since 2.0.1. Using it produces warning CS0618:

> Chuvadi.Sheets.Zip is obsolete and will be removed in the next major version. Use the Chuvadi.Zip package (namespace Chuvadi.Zip) instead.

It still works exactly as in 1.x, with its own `ZipFormatException`. New code should use Chuvadi.Zip, which is a separate package with its own guide (`docs/Chuvadi.Zip/GUIDE.md`). Zip password protection is not provided by either: classic ZipCrypto is broken and AES zip is not in the base class library. To protect a spreadsheet, encrypt the workbook (chapter 25).

31 Options reference

31.1 `XlsxReaderOptions`

Property	Default	Meaning
`TreatFirstRowAsHeaders`	`true`	Streaming reader: row 1 is headers, not data, and names the columns
`AutoDetectDates`	`true`	Numbers with a date or time format are returned as `DateTime`
`StrictXml`	`false`	Throw on malformed XML in optional parts (themes, charts) instead of skipping them
`IncludeStyledEmptyCells`	`false`	`Workbook.Load`: load empty cells that carry a style
`LoadImageData`	`true`	`Workbook.Load`: read picture bytes into memory
`DisplayCulture`	`null` (invariant, as Excel en-US)	Culture for display text
`MaxPartBytes`	`null`	Decompressed bytes per part
`MaxTotalUncompressedBytes`	`null`	Decompressed bytes for the package; also the size of an encrypted file
`MaxCompressionRatio`	`null`	Per entry larger than 1 MB
`MaxPackageEntries`	`null`	Zip entries
`MaxRowsPerSheet`	`null`	Rows read per sheet
`MaxSpinCount`	`500,000`	Key-derivation iterations accepted from an encrypted file
`CancellationToken`	none	Cancels opening, decryption and row reading
`ForUntrustedInput()`	—	256 MB per part, 1 GB total, 500:1, 20,000 entries, 1,048,576 rows

31.2 `XlsxWriterOptions`

Property	Default	Meaning
`UseInlineStrings`	`false`	Write strings in their cells instead of a shared strings table
`TempDirectory`	`Path.GetTempPath()`	Where the temporary files go (section 7.6)
`Encryption`	`null`	`EncryptionOptions` for a password-encrypted file
`DefaultDateFormat`	`yyyy-mm-dd`	Dates without a time and no style
`DefaultDateTimeFormat`	`yyyy-mm-dd hh:mm:ss`	Dates with a time and no style
`DefaultTimeSpanFormat`	`[h]:mm:ss`	`TimeSpan` values with no style

Property	Default	Meaning
`FullCalculationOnLoad`	`false`	Tell Excel to recalculate every formula on open

31.3 Other options

Type	Properties
`EncryptionOptions`	`Password` (required), `SpinCount` (100,000)
`AutoFitOptions`	`MinWidth`, `MaxWidth`, `MaxRowsToMeasure`, `ExtraWidth`, `AllowForFilterButton`, `Columns`, `Format`
`NumberFormatOptions`	`Culture`, `Date1904`
`ConditionalFormatOptions`	`Today`, `Date1904`
`SheetProtectionOptions`	`AllowSelectLockedCells`, `AllowSelectUnlockedCells`, `AllowFormatCells`, `AllowFormatColumns`, `AllowFormatRows`, `AllowInsertColumns`, `AllowInsertRows`, `AllowInsertHyperlinks`, `AllowDeleteColumns`, `AllowDeleteRows`, `AllowSort`, `AllowAutoFilter`, `AllowPivotTables`, `AllowEditObjects`, `AllowEditScenarios`
`PageSetup`	Chapter 13
`SheetView`	Chapter 12

32 Public API map

All types are in `Chuvadi.Sheets.Model`.

Area	Types
Model	`Workbook`, `Sheet`, `Cell`, `Range`, `Row`, `Column`, `RowIndexer`, `ColumnIndexer`, `ColumnRange`, `SheetFormat`, `DefinedName`, `DefinedNameCollection`, `LoadWarning`, `LoadWarningKind`
Streaming write	`XlsxWriter`, `SheetWriter`, `RowBuilder`, `RowFormat`, `ColumnFormat`, `XlsxWriterOptions`
One-line export	`XlsxExtensions`, `XlsxExport`, `XlsxExportConfig<T>`, `DataTableExportConfig`, `ExportConfigBase`, `ColumnAttribute`, `ColumnStyleAttribute`, `ColumnIgnoreAttribute`, `TableStyle`
Streaming read	`XlsxReader`, `SheetReader`, `RowReader`, `SheetInfo`, `SheetLayoutInfo`, `DefinedNameInfo`, `SheetVisibility`, `XlsxReaderOptions`, `XlsxReadExtensions`
Styles	`CellStyle`, `CellStyleBuilder`, `Border`, `BorderStyle`, `HorizontalAlign`, `VerticalAlign`, `UnderlineStyle`, `FontVerticalAlign`, `FontScheme`, `FillPattern`, `GradientFill`, `GradientStop`, `GradientType`, `ReadingOrder`
Style table (read)	`CellStyleTable`, `CellFormatEntry`, `FontEntry`, `FillEntry`, `BorderEntry`, `BorderEdge`, `AlignmentInfo`, `ProtectionInfo`, `NamedCellStyle`, `DifferentialFormat`, `ThemeInfo`, `ThemeFontScheme`, `ColorRef`, `ColorKind`
Display	`NumberFormatter`, `NumberFormatOptions`, `FormattedValue`
Formulas	`CellFormula`, `FormulaKind`
Rich text, links, comments	`RichText`, `RichTextRun`, `RichTextFont`, `Hyperlink`, `CellComment`, `CommentAnchor`
Views and printing	`SheetView`, `SheetPane`, `SheetSelection`, `PaneState`, `PaneKind`, `SheetViewMode`, `PageSetup`, `PageMargins`, `PageBreak`, `PaperSizeInfo`, `PageOrientation`, `PageOrder`, `PrintCellComments`, `PrintErrors`, `HeaderFooterText`
Validation	`DataValidation`, `DataValidationType`, `DataValidationErrorStyle`, `ComparisonOp`
Conditional formatting	`ConditionalFormat` (with `ColorScale`, `DataBar`), `ConditionalFormatRule`, `ConditionalRule`, `ConditionalRuleType`, `ConditionalFormatting`, `ConditionalValue`, `ConditionalValueType`, `ConditionalTimePeriod`, `ColorScaleInfo`, `DataBarInfo`, `DataBarAxisPosition`, `DataBarDirection`, `IconSetInfo`, `DifferentialStyle`
Evaluation	`ConditionalFormatEvaluator`, `ConditionalFormatOptions`, `ConditionalFormatResult`, `DataBarResult`, `IconResult`, `ICellValueSource`, `SheetValueSource`

Area	Types
Drawings and charts (read)	`DrawingObject`, `DrawingKind`, `DrawingAnchor`, `DrawingAnchorType`, `DrawingEditAs`, `DrawingPoint`, `PictureCrop`, `Emu`, `ChartInfo`, `ChartPlot`, `ChartSeries`, `ChartDataReference`, `ChartType`, `ChartSheet`, `HeaderFooterPicture`, `HeaderFooterSection`, `HeaderFooterPart`, `HeaderFooterPageKind`
Auto-fit	`AutoFitOptions`
Protection and encryption	`SheetProtectionOptions`, `EncryptionOptions`
Exceptions	`XlsxFormatException`, `XlsxPasswordRequiredException`, `XlsxLimitExceededException`

Every public member has XML documentation, shipped in the package as `Chuvadi.Sheets.xml`, so IntelliSense shows the details this guide leaves out.

33 Troubleshooting

Excel shows a number instead of a date. The cell was written as a number. Write a `DateTime` or `DateOnly` (they get `DefaultDateFormat`), or give the cell a style with a date format.

`InvalidOperationException: Cell 'X' is null or missing.` A typed getter met an empty cell. Use `TryGet<T>`, or check `row[col] is null` first.

A header name finds nothing. With `TreatFirstRowAsHeaders`, names come from row 1, matched case-insensitively. An unknown name reads like an empty cell: `row["Nmae"]` is `null`, and `GetString("Nmae")` throws the `InvalidOperationException` above. Check the header text, or switch headers off and use indexes.

Formulas show no value in a viewer. The library does not calculate. Write the result as the cached value, or set `FullCalculationOnLoad` so Excel recalculates on open.

A style is lost on load and save. Some properties are read but not written (chapter 20).

`XlsxFormatException` on a file Excel opens. Look at `InnerException`: it holds the parser's error. With `StrictXml` off, optional parts such as themes and charts are skipped, but a broken worksheet, shared strings table or relationship is reported. Please report a file Excel opens and the library refuses.

`XlsxPasswordRequiredException` on a file without a password. The file is encrypted. Excel encrypts some files with its built-in password `VelvetSweatshop` and opens them without asking; try that password, or ask the sender for the real one.

Opening an encrypted file is slow on Windows the first time. Antivirus scans it by trying Excel's default password at the file's spin count (chapter 25). Open from memory, or expect it once per file.

The streamed file is missing, or still the old file. Call `writer.Save()` before disposing the writer: a path save creates or replaces the file only when `Save` succeeds.

A `.<name>.<8 hex digits>.tmp` file next to the output. A path save was interrupted, for example the process was killed during `Save`. It is incomplete; delete it.

An encrypted file from an earlier version will not open. Before 2.0.1, encrypted files larger than about 7 MB were written incorrectly (chapter 25). Save them again with 2.0.1.

`AutoFitColumns must be called before the first row is written.` On the streaming writer, call it before `WriteHeader` and `WriteRow`.

A `MemoryStream` throws after `SaveTo` (2.0.0 and earlier). Those versions closed the stream after an unencrypted save; take the bytes with `ToArray()`, or update to 2.0.1, which leaves it open (section 6.4).

34 About this guide

34.1 Source and rendering

The guide's source is `docs/Chuvadi.Sheets/GUIDE.md`. The PDF is rendered from it with Chuvadi's own tools:

- **Chuvadi.Layout.Markup** lays out the Markdown;
- **Chuvadi.Layout.Pdf** writes the PDF, through Chuvadi.Pdf;
- **Chuvadi.Layout.Fonts** supplies Carlito for the text and Liberation Mono for the code, and **Chuvadi.Fonts** supplies LiPi Sans for the Tamil name. All are under the SIL Open Font License 1.1.

Chuvadi.Layout.Pdf is not yet part of the solution (it lives in `staging/`), so rendering needs the Chuvadi.Pdf packages in the local package folder, as for the other staged projects:

```
dotnet run -c Release --project docs\Chuvadi.Sheets\tools\GuideRenderer
```

That writes `docs/Chuvadi.Sheets/Chuvadi-Sheets-Guide-v2.0.pdf`. The run fails if the layout or the PDF writer reports any warning, such as a missing glyph.

34.2 Checked examples

Every C# example in this guide is compiled and run against the library's source before the PDF is published:

```
dotnet run -c Release --project docs\Chuvadi.Sheets\tools\GuideSnippets -- --extract
dotnet run -c Release --project docs\Chuvadi.Sheets\tools\GuideSnippets
```

The first command turns each C# block into a method (blocks that declare types go to file scope); the second runs them in order in a temporary folder and reports any failure with its line in `GUIDE.md`. Examples that only declare a local function, such as an upload handler, are compiled but not run.

34.3 History

- **2.0.1** (November 2026): updated for Chuvadi.Sheets 2.0.1 (section 4.5): streams left open, safe replace and encrypted temporary files, large encrypted files, header-name rules, overlapping column spans.
- **2.0** (November 2026): rebuilt from the library's README and its 2.0.0 API, replacing the 1.0 guide, whose source was lost. New: the `Chuvadi.Sheets.Model` namespace, one exception for malformed files, the `MaxPartBytes` marker, the obsolete zip API, and everything added from 1.1 to 1.8: reading styles, display text, formulas with cached results, page setup, auto-fit, row and column settings, views, conditional format evaluation, rich text, hyperlinks, comments, drawings and charts, read limits and cancellation.
- **1.0** (May 2026): the first guide, for Chuvadi.Sheets 1.0.

The library's full change history is in `src/Chuvadi.Sheets/CHANGELOG.md`.

34.4 Licence

Chuvadi.Sheets is released under the MIT License. Copyright (c) 2026 Arun Shiva B. The guide is part of the repository and under the same licence.